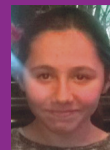




Christophe PICHAUD
Consultant sur les technologies Microsoft
christophepichaud@hotmail.com |
www.windowscpp.net



Pour Lisa.

Développer un IDE en C++

Partie 2

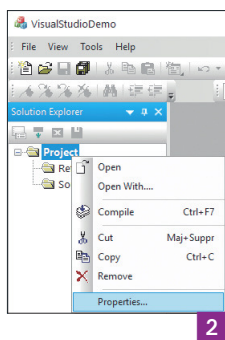
Construire un IDE est une aventure, et cela fait partie de ces applications dans lesquelles vous passez du temps pour valider l'ergonomie et découvrir les fonctionnalités qu'il vous manque. En un mot, je développe de l'intérieur. La première chose qu'il me manque c'est l'IntelliSense mais avec le temps je m'en passe. Comme dans Visual Studio je tape entièrement les mots, cela ne me manque pas trop. Après il y a un assistant comme MFC Class Wizard. Là, je m'appuie sur l'existant de mon code, car pour ajouter une commande et définir une fonction membre dans une classe, je peux le faire tout seul. Bref, l'analogie est très simple : au début c'est dur et puis avec le temps, on prend ses marques et on laisse parler l'expérience. Le mois dernier, je vous parlais d'un projet de réaliser un IDE léger. Ce mois-ci, je vous propose de découvrir les fonctionnalités qui ont été apportées au projet. Le projet est accessible sous GitHub ; si vous voulez jeter un coup d'œil aux sources, c'est le moment !

Montre-moi ton code

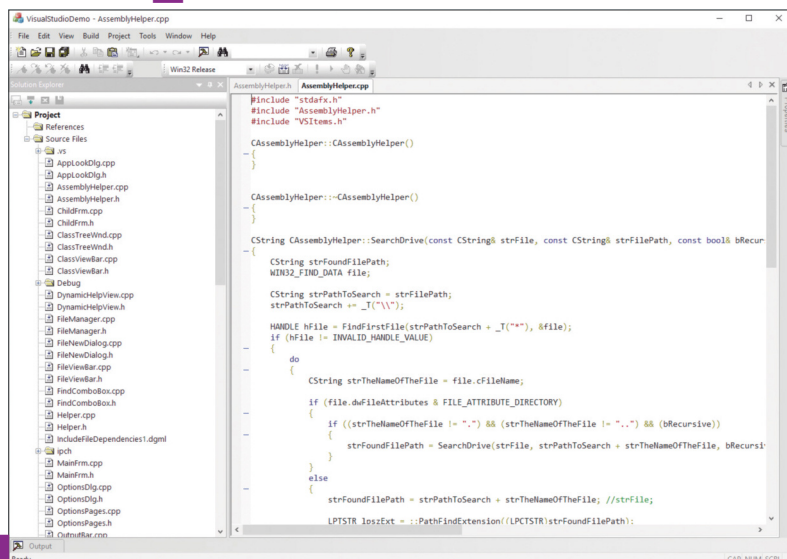
Comme le support de cet article est avant tout un développement concret, on a décidé avec François Tonic (le redac'chef) de mettre le code sur GitHub. Voici : l'URL : <https://github.com/ChristophePichaud/VSDemo>. Ce développement peut servir de base à la construction d'une application MDI qui implémente des panneaux accrochables (Docking Pane) et des toolbars flottantes. Vous verrez l'impressionnante fonctionnalité Full-Screen plus loin dans l'article.

Downloader un package binaire

Si vous n'avez pas de Visual Studio 2015 pour compiler les sources, récupérez une archive zip sur <http://www.windowscpp.com/Appz/VSDemo.zip>



2



1

Architecture de code

Pour comprendre comment est architecturé le code, il faut revenir aux fondamentaux. Cette application MFC est MDI donc cela veut dire que plusieurs fenêtres filles peuvent s'afficher sur la MainFrame. Si vous avez lu l'article du mois dernier, vous savez que la fenêtre qui contient le code source est implémentée par la librairie Open-Source Scintilla. Maintenant que l'éditeur de code est fait avec une classe dérivée de CView, on peut se concentrer sur les autres fonctionnalités.

Editer du code librement

La fonctionnalité « Open Folder » permet de visualiser tous les fichiers d'un répertoire et ainsi de pouvoir les éditer. C'est avec cette fonctionnalité que je fais du dog-fooding. Dans l'image ci-dessous, j'ai fait « Open Folder » de « D:\Dev\GitHub\VSDemo\VisualStudio Demo » qui est l'endroit où sont les sources de l'application. 1

La notion de projet et de solution

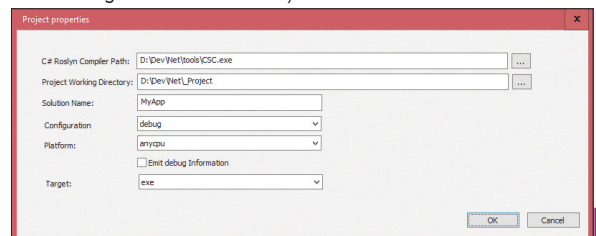
Dans VS, il peut y avoir plusieurs projets dans une solution. Ici, il n'y a qu'un projet principal actif et la solution est un fichier INI dans lequel je renseigne tous les paramètres du projet. Pour obtenir les propriétés du projet, il faut utiliser le pop-menu accessible en faisant bouton-droit sur l'item « Project » et choisir « Properties » : 2

Une boîte de dialogue permet de préciser les grandes caractéristiques du projet à savoir :

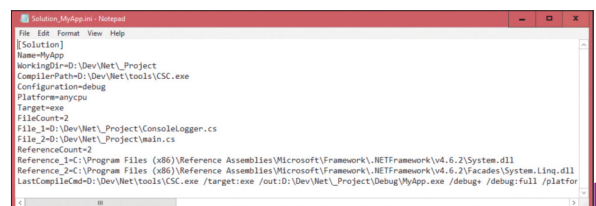
- Le chemin vers le compilateur Roslyn ;
- Le répertoire de travail du projet ;
- Le nom de la solution ;
- La configuration (Debug, Release) ;
- La plateforme (x86, x64, AnyCpu) ;
- La target (exe, dll). 3

Voici le fichier solution INI : 4

Le fichier contient explicitement le nom des fichiers du projet ainsi que des assemblies qui font office de références. A partir de ces informations, j'ai tout à disposition pour lancer le compilateur Roslyn et lui faire générer une assembly.



3



4

Rappels sur l'application

Le projet en l'état ressemble à ça... On y trouve un menu « File » riche : **5**

« New » permet de créer un fichier. « Add Existing » permet d'ajouter un fichier au projet. « Open Solution » permet d'ouvrir une solution. « Open Folder » permet de visualiser une arborescence de fichiers et de répertoires. C'est ma fonctionnalité préférée.

Par défaut, il y a toujours un projet. **6**

Un projet contient des fichiers et des références à des assemblages .NET, qu'elles proviennent du GAC ou du Framework 4.6.2. Pour ajouter un fichier au projet, il faut sélectionner « New ». A partir de là, une boîte propose de créer le fichier : **7**

Pour gérer les références, il faut faire « Properties » sur l'item References : **8 9**

Cette boîte permet de choisir des références, soit depuis le répertoire du Framework 4.6.2, soit depuis le GAC, soit via la boîte de dialogue commune Ouvrir pour sélectionner un fichier.

A partir de là, on dispose techniquement de toutes les composantes nécessaires à la création d'un projet et sa compilation avec les informations minimales. Voici le code qui permet de localiser les assemblages du framework ou dans le GAC :

```
CString CAssemblyHelper::GetWindowsDirectory()
{
    TCHAR IpszWinDir[MAX_PATH];
    ::GetWindowsDirectory(IpszWinDir, MAX_PATH);
    CString str = IpszWinDir;
    return str;
}

CString CAssemblyHelper::GetFrameworkPath()
{
    TCHAR IpszBuffer[255];
    DWORD dw = 255;
    ::GetEnvironmentVariable(_T("ProgramFiles(x86)"), IpszBuffer, dw);
```

```
CString strWinDir = IpszBuffer;
CString str;
str.Format(_T("%s\\Reference Assemblies\\Microsoft\\Framework\\.NETFramework\\v4.6.2"), strWinDir);
return str;
}

CString CAssemblyHelper::GetGACPath()
{
    CString strWinDir = CAssemblyHelper::GetWindowsDirectory();
    CString str;
    str.Format(_T("%s\\assembly"), strWinDir);
    return str;
}
```

Vous allez me dire, « ouaip, le Framework 4.6.2 est codé en dur ! ». Oui je sais mais ça va changer. Je donnerais la possibilité de choisir le Framework .NET dans une combo-box. Bref, comme vous le savez, le développement ça prend du temps et l'article doit être déposé très rapidement compte tenu des vacances de Noël donc certaines parties sont hard-codées. Vilain !

Les fonctionnalités de Recherche

CTRL+F lance la boîte de dialogue de Recherche : **10**

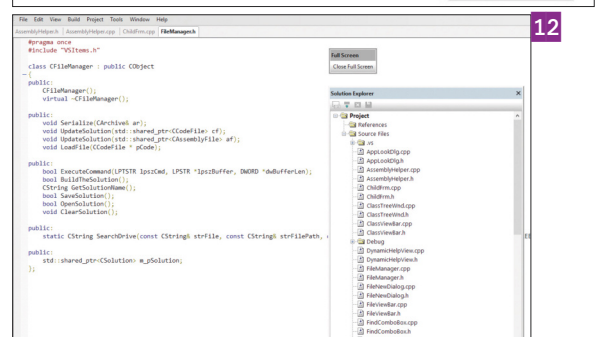
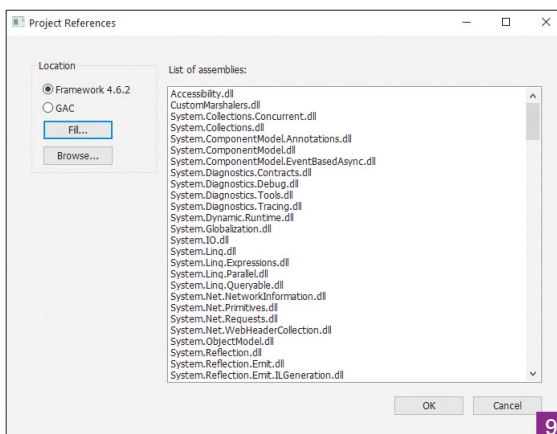
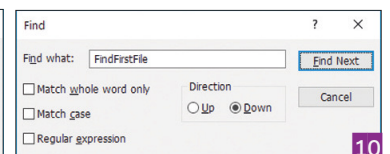
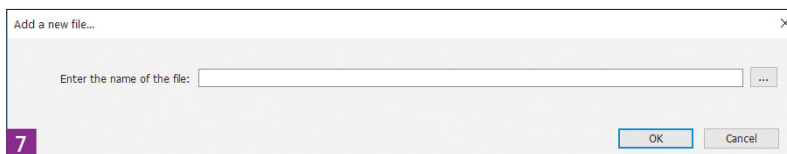
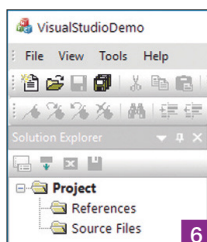
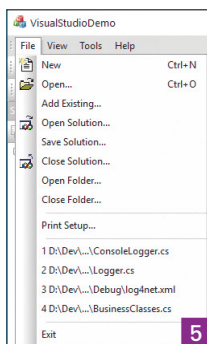
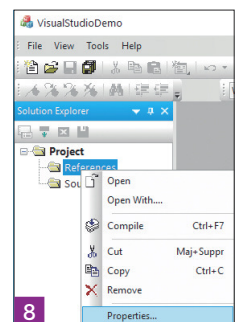
Elle fonctionne en *modeless* et surligne le code en arrière-plan. Cette fonctionnalité est très pratique.

La fonctionnalité « Full Screen »

Ajoutez ce bouton via le *customize* de la toolbar ; fonctionnalité disponible dans Build : **11**

Cette fonctionnalité permet d'avoir un large aperçu du code. C'est assez confortable. **12**

De plus, en demandant l'affichage du DockablePane « Solution Explorer » que l'on laisse en flottant, c'est beau !



Les fonctionnalités qui manquent

Il faut que je gère :

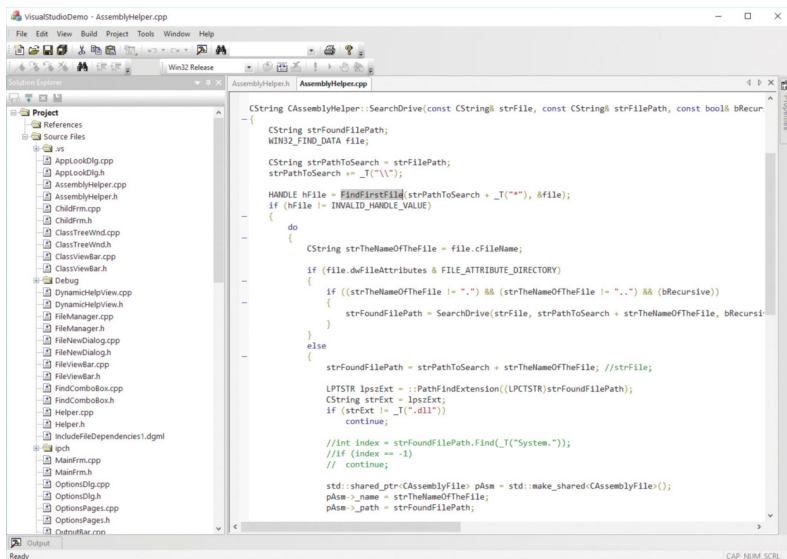
- La toolbar avec les combo-box Recherche et Target ;
- Le bouton Run qui lance le module si c'est un exe ;
- Le clean de la solution ;
- La gestion de plusieurs projets dans la solution ;
- Le passage du fichier de solution a peut-être autre chose que les fichiers INI ;
- La fonctionnalité « Find in Files ».

Bref, un développement propre prend du temps. Le fait de développer dans l'IDE requiert des fonctionnalités importantes et plein de petites choses qui font que le logiciel est de bonne qualité.

Conclusion

Pour rappel, cet IDE est une application que j'ai développée pour ma fille Lisa qui vient d'avoir 12 ans et j'ai commencé à lui enseigner les bases du C# : déclarer un int, une string, faire une boucle for, faire une opération mathématique complexe et l'exécuter 100 000 fois et vérifier que cela met moins d'une seconde ; bref les bases du code. Je dois avouer que mon défi est réussi car c'est utilisable. Maintenant, je vais lui faire découvrir les joies du code. Faire un foreach sur une collection, stocker des données dans un fichier. Bref, ma version personnelle « d'apprendre le code ». C'est tellement stimulant qu'elle s'y est prise au jeu. Elle a compris que l'ordinateur PC, cette grosse calculatrice qui vient avec Windows est pour les créateurs. Et donc, elle va s'éduquer via mon IDE léger et j'en suis fier. L'IDE est sous GitHub donc si vous voulez le customiser, faites-vous plaisir !

-



images en plus

